

Операционные системы

Лекция 13

Свопинг и виртуальная память

План лекции

- Мультипрограммирование со свопингом
- Виртуальная память: основные концепции, страничная, сегментная и сегментно-страничная организации виртуальной памяти
- Задачи управления виртуальной памятью: задача размещения, задача перемещения, задача преобразования адресов, задача замещения
- Подкачка
- Алгоритм замещения страниц

Виртуальная память

Необходимым условием для того, чтобы программа могла выполняться, является ее нахождение в оперативной памяти

Только в этом случае процессор может извлекать команды из памяти и интерпретировать их, выполняя заданные действия

Объем оперативной памяти, который имеется в компьютере, существенно сказывается на характере протекания-вычислительного процесса

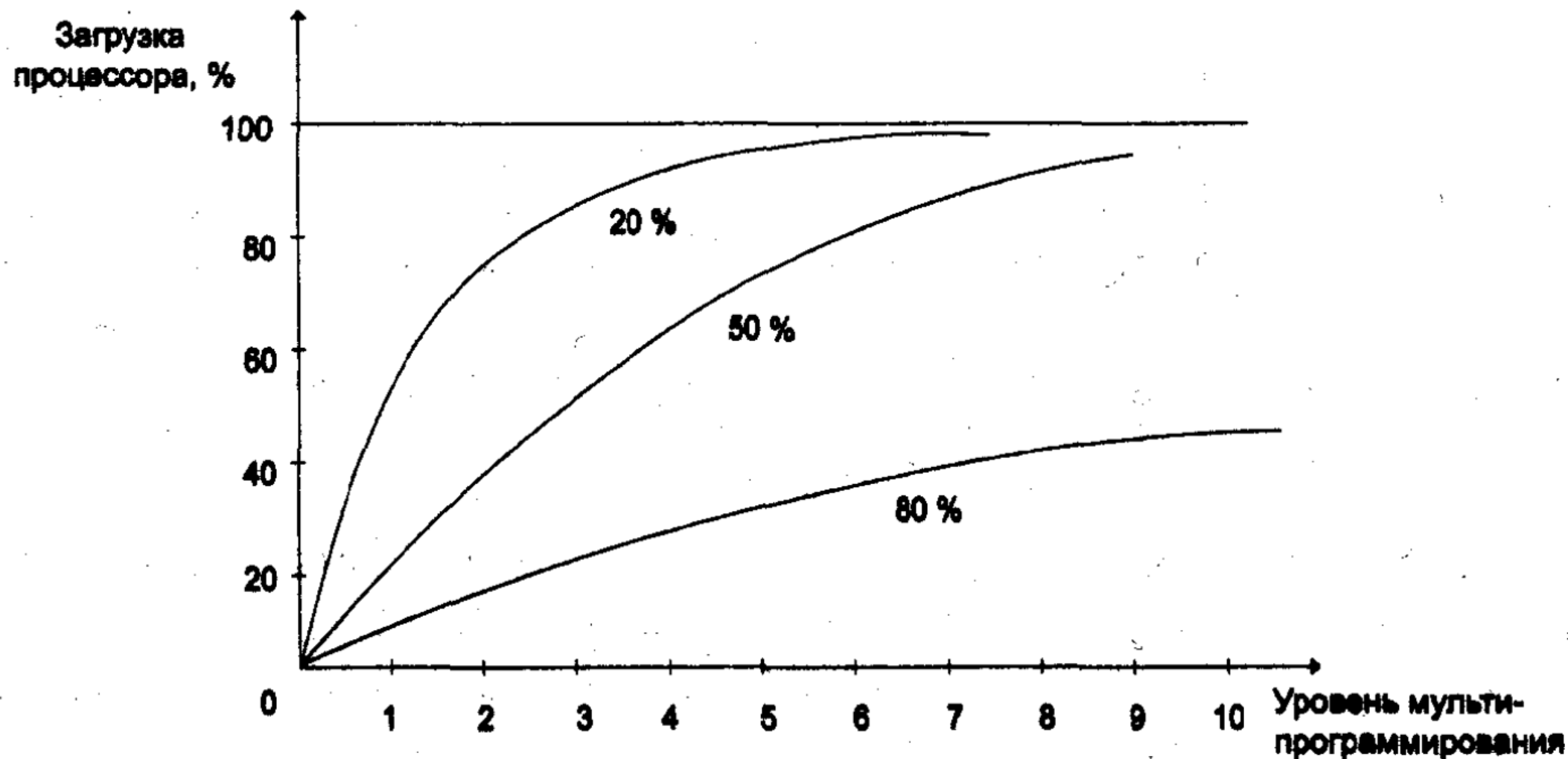
Он ограничивает число одновременно выполняющихся программ и размеры их виртуальных адресных пространств

Виртуальная память

В некоторых случаях, когда все задачи мультипрограммной смеси являются вычислительными (то есть выполняют относительно мало операций ввода-вывода, разгружающих центральный процессор), для хорошей загрузки процессора может оказаться достаточным всего 3-5 задач

Однако если вычислительная система загружена выполнением интерактивных задач, то для эффективного использования процессора может потребоваться уже несколько десятков, а то и сотен задач

Виртуальная память



Виртуальная память

На графике отображена зависимость коэффициента загрузки процессора от числа одновременно выполняемых процессов и доли времени, проводимого этими процессами в состоянии ожидания ввода-вывода

Большое количество задач, необходимое для высокой загрузки процессора, требует большого объема оперативной памяти

В условиях, когда для обеспечения приемлемого уровня мультипрограммирования имеющейся оперативной памяти недостаточно, был предложен метод организации вычислительного процесса, при котором образы некоторых процессов целиком или частично временно выгружаются на диск

Виртуальная память

В мультипрограммном режиме помимо активного процесса, то есть процесса, коды которого в настоящий момент интерпретируются процессором, имеются приостановленные процессы, находящиеся в ожидании завершения ввода-вывода или освобождения ресурсов, а также процессы в состоянии готовности, стоящие в очереди к процессору

Образы таких неактивных процессов могут быть **временно**, до следующего цикла активности, **выгружены на диск**

Несмотря на то что коды и данные процесса отсутствуют в оперативной памяти, ОС «знает» о его существовании и в полной мере учитывает это при распределении процессорного времени и других системных ресурсов

Виртуальная память

К моменту, когда подходит очередь выполнения выгруженного процесса, его образ возвращается с диска в оперативную память

Если при этом обнаруживается, что свободного места в оперативной памяти не хватает, то на диск выгружается другой процесс

Такая **подмена (виртуализация) оперативной памяти** дисковой памятью позволяет повысить уровень мультипрограммирования – объем оперативной памяти компьютера в этом случае не столь жестко ограничивает количество одновременно выполняемых процессов, поскольку суммарный объем памяти, занимаемой образами этих процессов, может существенно превосходить имеющийся объем оперативной памяти

Виртуальная память

Виртуальным называется ресурс, свойства которого с точки зрения пользователя или пользовательской программы отличаются от реальных свойств этого ресурса

В данном случае в распоряжение прикладного программиста предоставляется виртуальная оперативная память, размер которой намного превосходит всю имеющуюся в системе реальную оперативную память

Пользователь пишет программу, а транслятор, используя виртуальные адреса, переводит ее в машинные коды так, как будто в распоряжении программы имеется однородная оперативная память большого объема

Виртуальная память

В действительности же все коды и данные, используемые программой, хранятся на дисках и только при необходимости загружаются в реальную оперативную память

Понятно, однако, что работа такой «оперативной памяти» происходит значительно медленнее

Виртуализация оперативной памяти осуществляется совокупностью программных модулей ОС и аппаратных схем процессора и подразумевает решение следующих задач:

- Размещение данных в запоминающих устройствах разного типа, например, часть кодов программы – в оперативной памяти, а часть – на диске
- Выбор образов процессов или их частей для перемещения из оперативной памяти на диск и обратно
- Перемещение по мере необходимости данных между памятью и диском
- Преобразование виртуальных адресов в физические

Виртуальная память

Очень важно то, что все действия по организации совместного использования диска и оперативной памяти – выделение места для перемещаемых фрагментов, настройка адресов, выбор кандидатов на загрузку и выгрузку – осуществляются операционной системой и аппаратурой процессора автоматически, без участия программиста и никак не сказываются на логике работы приложений

Виртуальная память

Уже достаточно давно пользователи столкнулись с проблемой размещения в памяти программы, размер которой превышает имеющуюся в наличии свободную память

Одним из первых решений было разбиение программы на части, называемые оверлеями. Когда первый оверлей заканчивал свое выполнение, он вызывал другой оверлей. Все оверлеи хранились на диске и перемещались между памятью и диском средствами операционной системы на основании явных директив программиста, содержащихся в программе

Этот способ, несмотря на внешнее сходство, имеет принципиальное отличие от виртуальной памяти, заключающееся в том, что разбиение программы на части и планирование их загрузки в оперативную память должны были выполняться заранее программистом во время написания программы

Свопинг и виртуальная память

Различают два достаточно близких подхода к виртуализации памяти:

- **Свопинг (swapping)** – образы процессов выгружаются на диск и возвращаются в оперативную память целиком
- **Виртуальная память (virtual memory)** – между оперативной памятью и диском перемещаются части (сегменты, страницы и т. п.) образов процессов

Свопинг представляет собой частный случай виртуальной памяти и, следовательно, – это более простой в реализации способ совместного использования оперативной памяти и диска

Виртуальная память

Однако ему свойственна избыточность: когда ОС решает активизировать процесс, для его выполнения, как правило, не требуется загружать в оперативную память все его сегменты полностью – достаточно загрузить небольшую часть кодового сегмента с подлежащей выполнению инструкцией и частью сегментов данных, с которыми работает эта инструкция, а также отвести место под сегмент стека

Аналогично, при освобождении памяти для загрузки нового процесса очень часто вовсе не требуется выгружать другой процесс на диск целиком, достаточно вытеснить на диск только часть его образа

Перемещение избыточной информации замедляет работу системы, а также приводит к неэффективному использованию памяти. Кроме того, системы, поддерживающие свопинг, имеют еще один очень существенный недостаток: ***они не способны загрузить для выполнения процесс, виртуальное адресное пространство которого превышает имеющуюся в наличии свободную память***

Виртуальная память

Именно из-за указанных недостатков свопинг как основной механизм управления памятью почти не используется в современных ОС

На смену ему пришел более совершенный механизм **виртуальной памяти**, который, как уже было сказано, заключается в том, что при нехватке места в оперативной памяти на диск выгружаются только части образов процессов

Однако в некоторых современных ОС, например версиях Unix, основанных на коде SVR4, механизм свопинга используется как дополнительный к виртуальной памяти и включается только при серьезных перегрузках системы

Виртуальная память: виды

Ключевой проблемой виртуальной памяти, возникающей в результате многократного изменения местоположения в оперативной памяти образов процессов или их частей, является преобразование виртуальных адресов в физические

Решение этой проблемы, в свою очередь, зависит от того, какой способ структуризации виртуального адресного пространства принят в данной системе управления памятью

Все множество реализаций виртуальной памяти может быть представлено тремя классами: **сегментным, страничным и сегментно-страничным**

Виртуальная память: виды

В случае **страничной виртуальной памяти** перемещение данных между памятью и диском организуется страницами – частями виртуального адресного пространства фиксированного и сравнительно небольшого размера

Сегментная виртуальная память предусматривает перемещение данных сегментами – частями виртуального адресного пространства произвольного размера, полученными с учетом смыслового значения данных

В **сегментно-страничной виртуальной памяти** используется двухуровневое деление: виртуальное адресное пространство делится на сегменты, а затем сегменты делятся на страницы. Единицей перемещения данных здесь является страница. Этот способ управления памятью объединяет в себе элементы обоих предыдущих подходов

Свопинг и подкачка

Для временного хранения сегментов и страниц на диске отводится либо специальная область, либо специальный файл, которые во многих ОС по традиции продолжают называть **областью, или файлом, свопинга**, хотя перемещение информации между оперативной памятью и диском осуществляется уже не в форме полного замещения одного процесса другим, а частями

Другое популярное название этой области – **страничный файл (page file, или paging file)** или же **файл подкачки**. Текущий размер страничного файла является важным параметром, оказывающим влияние на возможности операционной системы: чем больше страничный файл, тем больше приложений может одновременно выполнять ОС (при фиксированном размере оперативной памяти)

Свопинг и подкачка

Однако необходимо понимать, что увеличение числа одновременно работающих приложений за счет увеличения размера страничного файла замедляет их работу, так как значительная часть времени при этом тратится на перекачку кодов и данных из оперативной памяти на диск и обратно

Размер страничного файла в современных ОС является настраиваемым параметром, который выбирается администратором системы для достижения компромисса между уровнем мультипрограммирования и быстродействием системы

Так, Microsoft советует иметь файл подкачки размера 1.5-2х от размера доступной оперативной памяти

Страничное распределение

Теперь же рассмотрим как устроены три ранее упомянутых способа организации виртуальной памяти

Первым рассмотрим страничный принцип организации виртуальной памяти

При страничном распределении памяти виртуальное адресное пространство каждого процесса делится на части одинакового, фиксированного для данной системы размера, называемые **виртуальными страницами** (virtual pages)

В общем случае, размер виртуального адресного пространства процесса не кратен размеру страницы, поэтому последняя страница каждого процесса дополняется фиктивной областью

Виртуальные
страницы

Виртуальное адресное
пространство процесса 1

0
1
2
3
4

Виртуальные
страницы

Виртуальное адресное
пространство процесса 2

0
1
2
3
4
5

Таблица страниц
процесса 1

	N _{стр.с}	Упр. инф.
0	5	
1	ВП	
2	ВП	
3	10	
4	2	

Таблица страниц
процесса 2

	N _{стр.с}	Упр. инф.
	8	
	ВП	
	ВП	
	ВП	
	ВП	
	11	

Физическая память

	0
	1
Стр. 4, пр. 1	2
	3
	4
Стр. 0, пр. 1	5
	6
Стр. 0, пр. 2	7
	8
Стр. 3, пр. 1	9
Стр. 5, пр. 2	10
	11
	12
⋮	

Страничный
обмен



Страничное распределение

Вся оперативная память машины также делится на части такого же размера, называемые **физическими страницами** (или блоками, или кадрами)

Размер страницы выбирается равным степени двойки: 512, 1024, 4096 байт и т. д., Это позволяет упростить механизм преобразования адресов

При создании процесса ОС загружает в оперативную память несколько его виртуальных страниц (начальные страницы кодового сегмента и сегмента данных). Копия всего виртуального адресного пространства процесса находится на диске. Смежные виртуальные страницы не обязательно располагаются в смежных физических страницах

Страничное распределение

Для каждого процесса операционная система создает **таблицу страниц** – информационную структуру, содержащую записи обо всех виртуальных страницах процесса

Дескриптор страницы – это отдельная запись таблицы. Она включает следующую информацию:

- Номер физической страницы, в которую загружена данная виртуальная страница
- Признак присутствия, устанавливаемый в единицу, если виртуальная страница находится в оперативной памяти
- Признак модификации страницы, который устанавливается в единицу всякий раз, когда производится запись по адресу, относящемуся к данной странице
- Признак обращения к странице, называемый также **битом доступа**, который устанавливается в единицу при каждом обращении по адресу, относящемуся к данной странице

Страничное распределение

Признаки присутствия, модификации и обращения в большинстве моделей современных процессоров устанавливаются схемами процессора аппаратно при выполнении операции с памятью

Информация из таблиц страниц используется для решения вопроса о необходимости перемещения той или иной страницы между памятью и диском, а также для преобразования виртуального адреса в физический

Сами таблицы страниц, так же как и описываемые ими страницы, размещаются в оперативной памяти. Адрес таблицы страниц включается в контекст соответствующего процесса. При активизации очередного процесса операционная система загружает адрес его таблицы страниц в специальный регистр процессора (CR3)

Страничное распределение

При каждом обращении к памяти выполняется поиск номера виртуальной страницы, содержащей требуемый адрес, затем по этому номеру определяется нужный элемент таблицы страниц и из него извлекается описывающая страницу информация (возможность кеширования в данном описании не учитывается)

Далее анализируется признак присутствия, и если данная виртуальная страница находится в оперативной памяти, то выполняется преобразование виртуального адреса в физический, то есть виртуальный адрес заменяется указанным в записи таблицы физическим адресом

Страничное распределение

Если же нужная виртуальная страница в данный момент выгружена на диск, то происходит так называемое **страничное прерывание**

Выполняющийся процесс переводится в состояние ожидания и активизируется другой процесс из очереди процессов, находящихся в состоянии готовности

Параллельно программа обработки страничного прерывания находит на диске требуемую виртуальную страницу (для этого операционная система должна помнить положение вытесненной страницы в страничном файле диска) и пытается загрузить ее в оперативную память

Страничное распределение

Если в памяти имеется свободная физическая страница, то загрузка выполняется немедленно, если же свободных страниц нет, то на основании принятой в данной системе стратегии замещения страниц решается вопрос о том, какую страницу следует выгрузить из оперативной памяти

После того как выбрана страница, которая должна покинуть оперативную память, обнуляется ее бит присутствия и анализируется ее признак модификации

Если выталкиваемая страница за время последнего пребывания в оперативной памяти была модифицирована, то ее новая версия должна быть переписана на диск

Страничное распределение

Если нет, то принимается во внимание, что на диске уже имеется предыдущая копия этой виртуальной страницы, и никакой записи на диск не производится

Физическая страница объявляется свободной. Из соображений безопасности в некоторых системах освобождаемая страница обнуляется, с тем чтобы невозможно было использовать содержимое выгруженной страницы

Для хранения информации о положении вытесненной страницы в страничном файле ОС может использовать поля таблицы страниц или же другую системную структуру данных (например, дескриптор сегмента при сегментно-страничной организации виртуальной памяти)

Страничное распределение

Виртуальный адрес яри страничном распределении может быть представлен в виде пары (p, s_v) , где p – порядковый номер виртуальной страницы процесса (нумерация страниц начинается с 0), а s_v – смещение в пределах виртуальной страницы

Физический адрес также может быть представлен в виде пары (n, s_f) , где n – номер физической страницы, а s_f – смещение в пределах физической страницы

Задача подсистемы виртуальной памяти состоит в отображении пары (p, s_v) на (n, s_f)

Страничное распределение

Прежде чем приступить к рассмотрению схемы преобразования виртуального адреса в физический, остановимся на **двух базисных свойствах** страничной организации:

Первое из них состоит в том, что объем страницы выбирается равным степени двойки – 2^k

Второе свойство заключается в том, что в пределах страницы непрерывная последовательность виртуальных адресов однозначно отображается на непрерывную последовательность физических адресов, а, значит, смещения в виртуальном и физическом адресах s_v и s_f равны между собой

Страничное распределение

Из первого свойства следует, что смещение s может быть получено простым отделением k младших разрядов в двоичной записи адреса, а оставшиеся старшие разряды адреса представляют собой двоичную запись номера страницы (при этом неважно, является страница виртуальной или физической)

Например, если размер страницы составляет 1 Кбайт (2^{10}), то из двоичной записи адреса $5071_8 - 101\ 000\ 111\ 001_2$ можно определить, что он принадлежит странице, номер которой в двоичном выражении равен 10_2 и смещен относительно ее начала на $1\ 000\ 111\ 001_2$ байт

Страничное распределение

00 0 000 000 000	Начало 0-й страницы
00 0 000 000 001	
00 0 000 000 010	
.....	
00 1 111 111 111	Конец 0-й страницы
01 0 000 000 000	Начало 1-й страницы
01 0 000 000 001	
.....	
01 1 111 111 111	Конец 1-й страницы
10 0 000 000 000	Начало 2-й страницы
10 0 000 000 001	
.....	
101 000 111 001	Исходный виртуальный адрес
.....	
10 1 111 111 111	Конец 2-й страницы
11 0 000 000 000	Начало 3-й страницы
.....	

Страничное распределение

Из рисунка хорошо видно, что номер страницы и ее начальный адрес легко могут быть получены один из другого дополнением или отбрасыванием k нулей, соответствующих смещению

Именно по этой причине часто говорят, что таблица страниц содержит начальный физический адрес страницы в памяти (а не номер физической страницы), хотя на самом деле в таблице указаны только старшие разряды адреса

Начальный адрес страницы называется **базовым адресом**

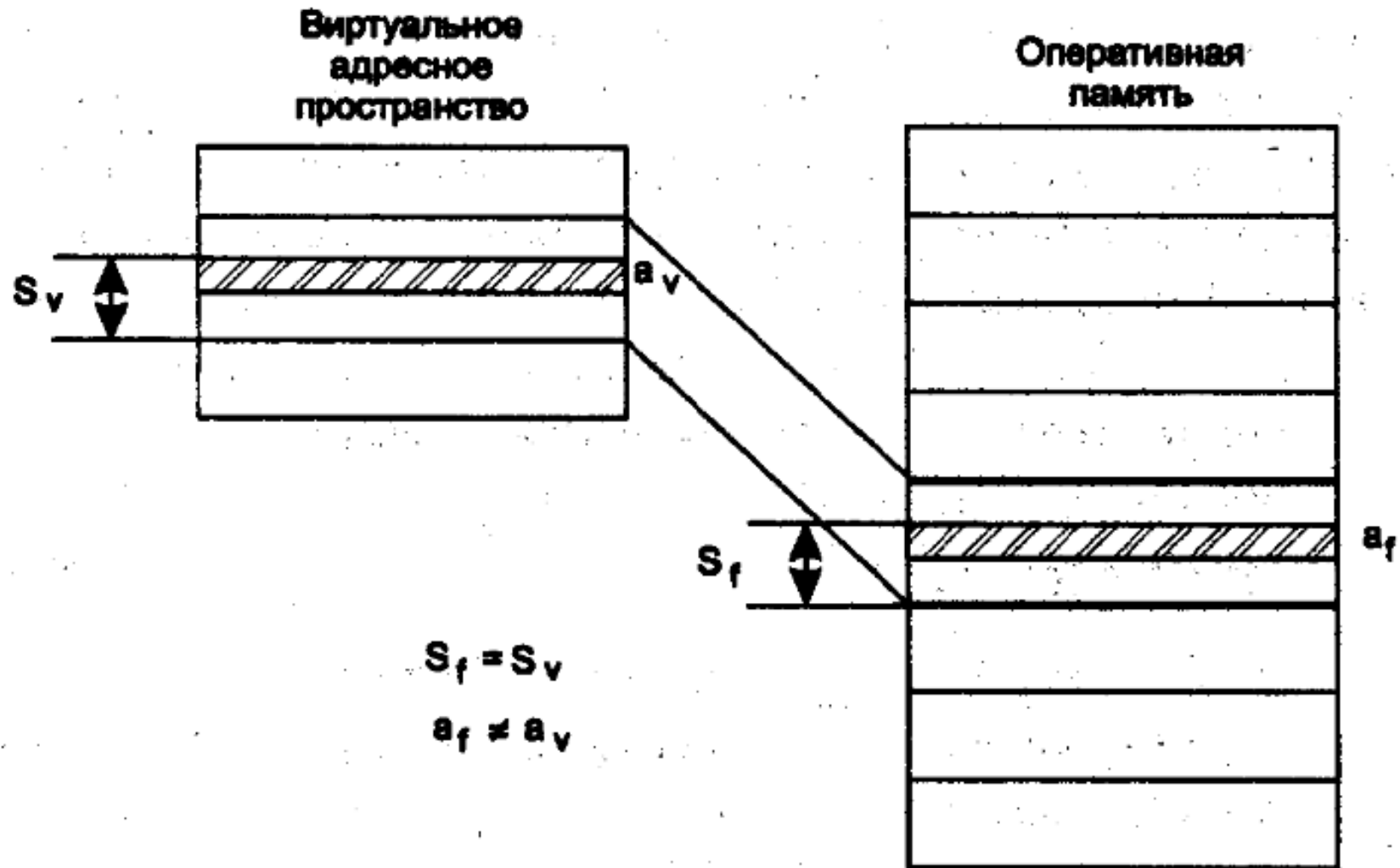
Страничное распределение

Из второго свойства следует простая схема преобразования виртуального адреса в физический

Младшие разряды физического адреса, соответствующие смещению, получаютcя переносом такого же количества младших разрядов из виртуального адреса

Старшие разряды физического адреса, соответствующие номеру физической страницы, определяются из таблицы страниц, в которой указывается соответствие виртуальных и физических страниц

Страничное распределение



Страничное распределение

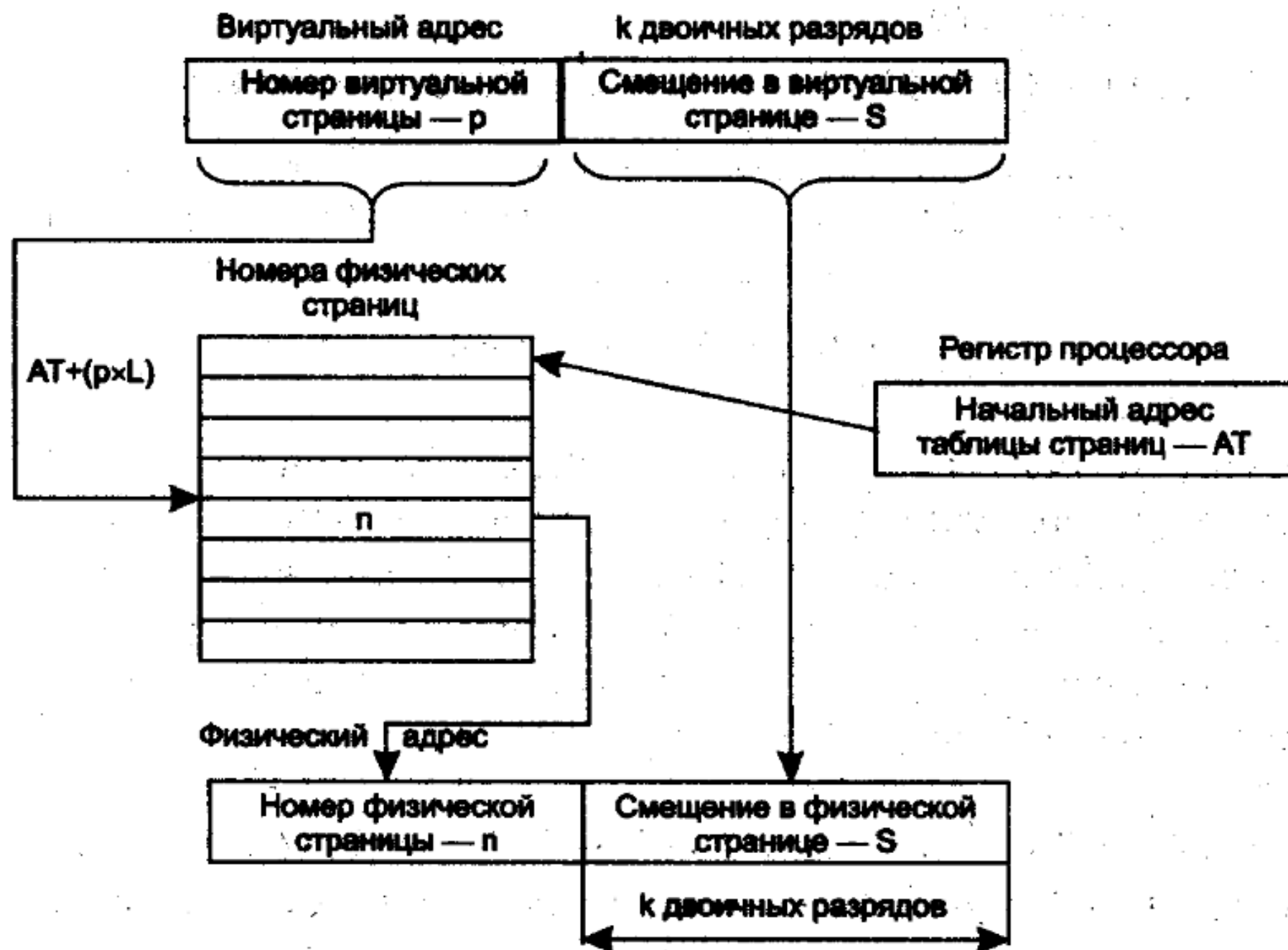
Итак, пусть произошло обращение к памяти по некоторому виртуальному адресу. Аппаратными схемами процессора выполняются следующие действия:

1. Из специального регистра процессора извлекается адрес АТ таблицы страниц активного процесса. На основании начального адреса таблицы страниц, номера виртуальной страницы p (старшие разряды виртуального адреса) и длины отдельной записи в таблице страниц L (системная константа) определяется адрес нужного дескриптора в таблице страниц:

$$a = AT + (p * L)$$

2. Из этого дескриптора извлекается номер соответствующей физической страницы – n

3. К номеру физической страницы присоединяется смещение s (младшие разряды виртуального адреса)



Страничное распределение

Страничное распределение памяти *может быть* реализовано в упрощенном варианте, *без выгрузки страниц на диск*

В этом случае все виртуальные страницы всех процессов постоянно находятся в оперативной памяти

Такой вариант страничной организации, хотя и не предоставляет пользователю преимуществ работы с виртуальной памятью большого объема, сохраняет другое достоинство страничной организации – позволяет успешно бороться с фрагментацией физической памяти

Свопинг и виртуальная память

Действительно, во-первых, программу можно разбить на части и загрузить в разрозненные участки свободной памяти, во-вторых, при загрузке виртуальных страниц никогда не образуется неиспользуемых остатков, так как размеры виртуальных и физических страниц совпадают

Такой режим работы системы управления памятью используется в некоторых специализированных ОС, когда требуется высокая реактивность системы и способность выполнять переменный набор приложений (пример – ОС семейства Novell NetWare 3.x и 4.x)

Оптимизация страничной памяти

При выполнении типичной машинной команды происходит в среднем 3-4 обращения к памяти (выборка команды, извлечение операндов, запись результата)

В системе с виртуальной страничной памятью при каждом из этих обращений необходимо выполнить либо преобразование виртуального адреса в физический, либо обработку страничного прерывания

Время выполнения этих операций в значительной степени влияет на общую производительность вычислительной системы, поэтому столь большое внимание разработчиков уделяется оптимизации виртуальной памяти

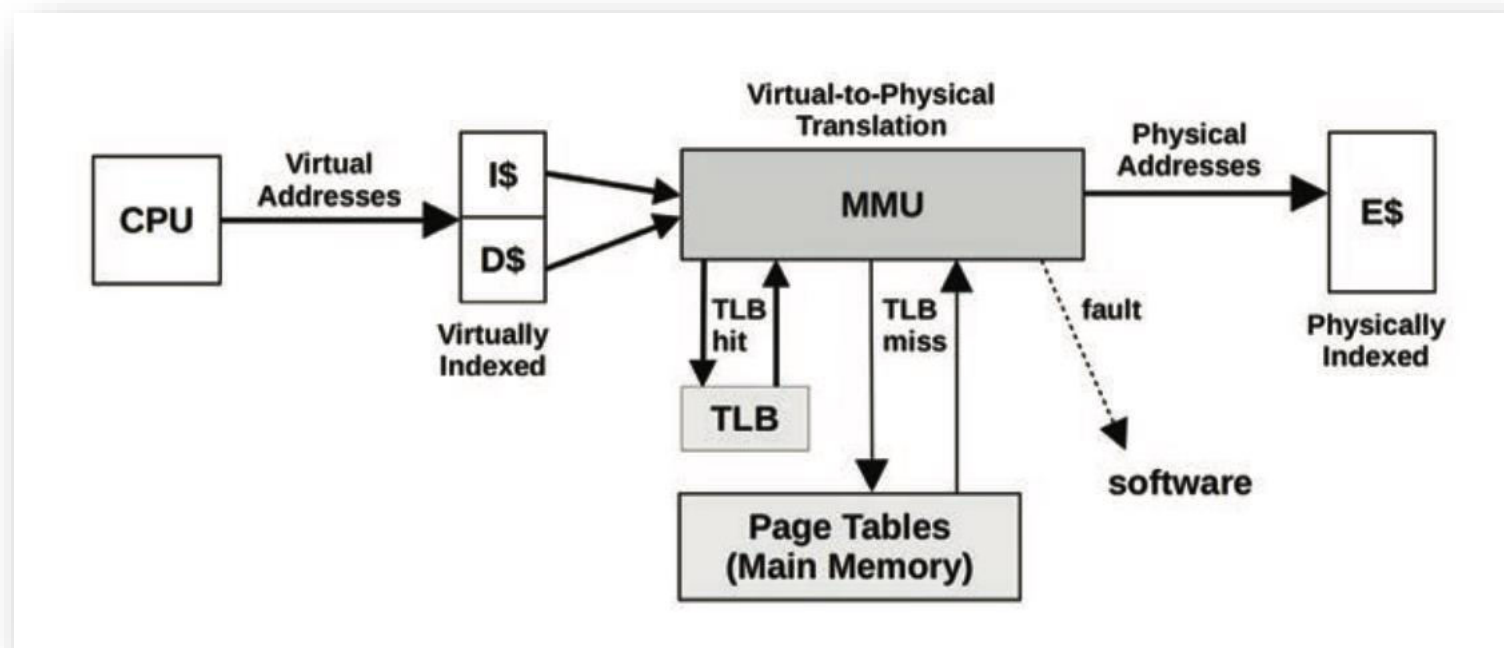
Оптимизация страничной памяти

Накладные расходы системы на поддержание страничной виртуальной памяти могут быть уменьшены за счет:

- ***Аппаратного способа преобразования виртуального адреса в физический***
- ***Снижения частоты страничных прерываний*** путем оптимизации размера страницы, а также выбора страниц на загрузку/выгрузку

За аппаратное преобразование виртуальных адресов а физические отвечает такое устройство как **блок управления памятью (Memory Management Unit (MMU))**

Оптимизация страничной памяти



Оптимизация страничной памяти

Эффективность аппаратного получения физического адреса по виртуальному может быть повышена, если размер страницы выбрать равным степени двойки. В этом случае двоичная запись адреса легко разделяется на номер страницы и смещение, а значит, в процедуре преобразования адресов более длительную операцию сложения можно заменить операцией присоединения (конкатенации)

Применяются и другие способы ускорения преобразования, такие, например, как кэширование таблицы страниц – хранение наиболее активно используемых записей в быстродействующих запоминающих устройствах, в частности в регистрах процессора

Оптимизация страничной памяти

Другим важным фактором, влияющим на производительность системы, является частота страничных прерываний, на которую, в свою очередь, влияют размер страницы и принятые в данной системе правила выбора страниц для выгрузки и загрузки

При неправильно выбранной стратегии замещения страниц могут возникать ситуации, когда система тратит большую часть времени впустую на подкачку страниц из оперативной памяти на диск и обратно

Оптимизация страничной памяти

При выборе страницы на выгрузку могут быть использованы различные критерии, смысл которых сводится к одному: на диск выталкивается страница, к которой в будущем, начиная с данного момента, дольше всего не будет обращений

Поскольку точно предсказать ход вычислительного процесса невозможно, то невозможно точно определить страницу, подлежащую выгрузке. В таких условиях решение принимается на основе неких эмпирических критериев, часто основывающихся на предположении об инерционности вычислительного процесса

Оптимизация страничной памяти

Так, например, из того, что страница не использовалась долгое время, делается вывод о том, что она, скорее всего, не будет использоваться и в ближайшее время

Однако привлечение критериев такого рода не исключает ситуаций, когда сразу после выгрузки страницы к ней происходит обращение и ее снова приходится загружать в память

Вероятность таких «напрасных» перемещений настолько велика, что в некоторых реализациях виртуальной памяти вообще отказываются от количественных критериев и предпочитают случайный выбор, при котором на диск выгружается первая попавшаяся страница

Возникающее при этом некоторое увеличение интенсивности страничного обмена компенсируется снижением вычислительных затрат на поддержание и анализ критерия выборки страниц на выгрузку

Оптимизация страничной памяти

Наиболее популярным критерием выбора страницы на выгрузку является число обращений к ней за последний период времени. Вычисление этого критерия происходит следующим образом:

Операционная система ведет для каждой страницы программный счетчик. Значения счетчиков определяются значениями признаков доступа. Всякий раз, когда происходит обращение к какой-либо странице, процессор устанавливает в единицу признак доступа в относящейся к данной странице записи таблицы страниц. ОС периодически просматривает признаки доступа всех страниц во всех существующих в данный момент записях таблицы страниц. Если какой-либо признак оказывается равным 1 (было обращение), то система сбрасывает его в 0, увеличивая при этом на единицу значение связанного с этой страницей счетчика обращений

Когда возникает необходимость удалить какую-либо страницу из памяти, ОС находит страницу, счетчик обращений которой имеет наименьшее значение. Для того чтобы критерий учитывал интенсивность обращений за последний период, ОС с соответствующей периодичностью обнуляет все счетчики

Оптимизация страничной памяти

Интенсивность страничного обмена может быть также снижена в результате так называемой **упреждающей загрузки**, в соответствие с которой при возникновении страничного прерывания в память загружается не одна страница, содержащая адрес обращения, а сразу несколько прилегающих к ней страниц

Здесь используется эмпирическое правило: ***если обращение произошло по некоторому адресу, то велика вероятность того, что следующие обращения произойдут по соседним адресам***

Оптимизация страничной памяти

Резервом повышения производительности системы является также правильный выбор размера страницы. Каким же должен быть оптимальный размер страницы?

С одной стороны, чтобы уменьшить частоту страничных прерываний, следовало бы увеличивать размер страницы. С другой стороны, если страница велика, то велика и фиктивная область в последней виртуальной странице каждого процесса

Если учесть, что в среднем в каждом процессе фиктивная область составляет половину страницы, то в сумме при большом объеме страницы потери могут составить существенную величину

Оптимизация страничной памяти

Из приведенных соображений следует, что выбор размера страницы является сложной оптимизационной задачей, требующей учета многих факторов. На практике же разработчики ОС и процессоров ограничиваются неким рациональным решением, пригодным для широкого класса вычислительных систем

Типичный размер страницы составляет несколько килобайтов, например, наиболее распространенные процессоры Pentium компании Intel, а также операционные системы, устанавливаемые на компьютерах с этими процессорами, поддерживают страницы размером 4096 байт (4 Кбайт)

Оптимизация страничной памяти

Размер страницы влияет также на количество записей в таблицах страниц. Чем меньше страница, тем более объемными являются таблицы страниц процессов и тем больше места они занимают в памяти

Учитывая, что в современных процессорах максимальный объем виртуального адресного пространства процесса, как правило, не меньше 4 Гбайт (2^{32}), то при размере страницы 4 Кбайт (2^{12}) и длине записи 4 байт для хранения таблицы страниц может потребоваться 4 Мбайт памяти!

Двухуровневая страничная память

Выходом в такой ситуации является хранение в памяти только той части таблицы страниц, которая активно используется в данный период времени – так как сама таблица страниц хранится в таких же страницах физической памяти, что и описываемые ею страницы, то принципиально возможно временно вытеснять часть таблицы страниц из оперативной памяти

Именно такой результат может быть достигнут путем более сложной структуризации виртуального адресного пространства – **многоуровневые таблицы страниц**

Двухуровневая страничная память

При двухуровневом страничном распределении памяти множество виртуальных адресов процесса делится на разделы, а разделы делятся на страницы

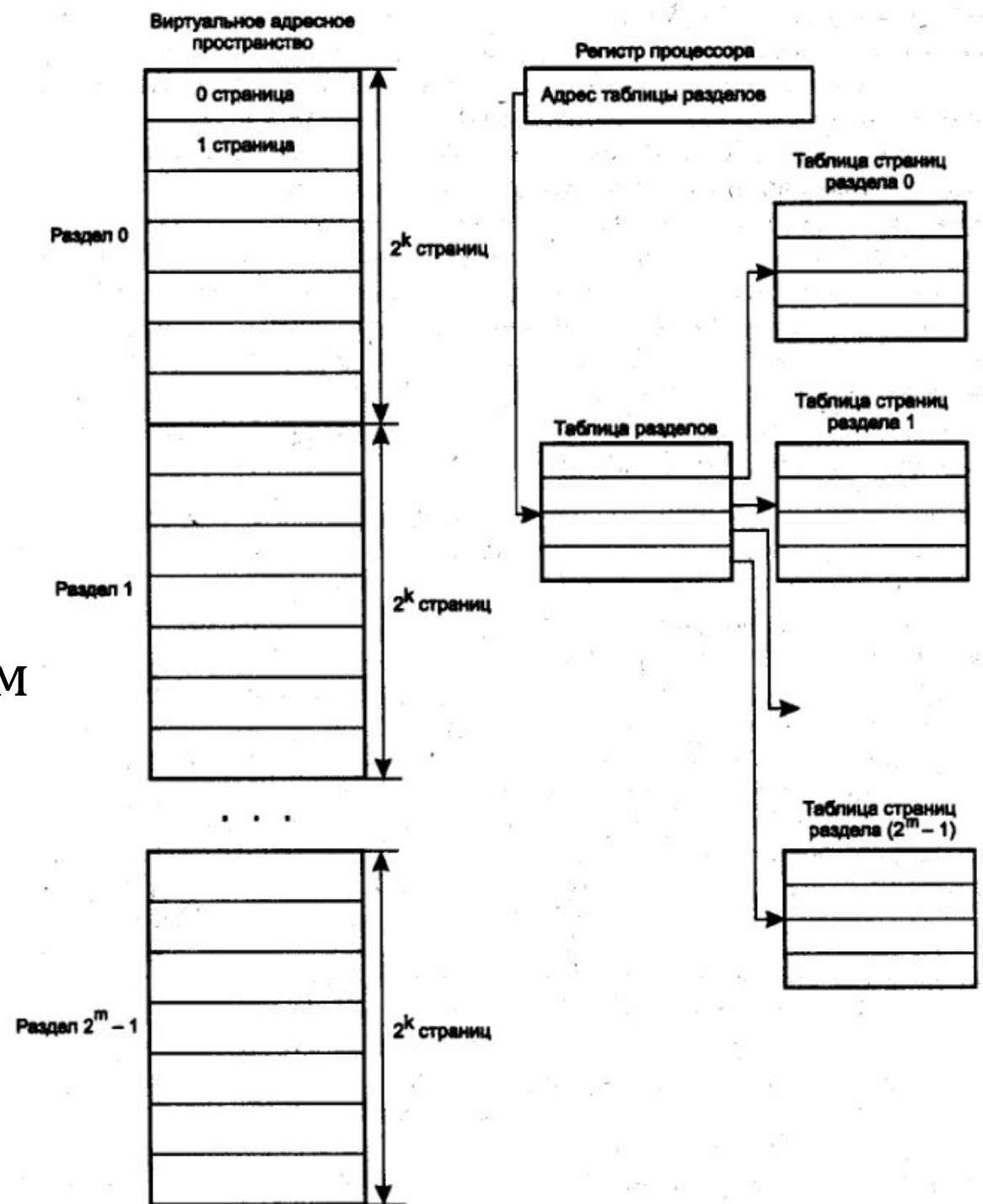
Все страницы имеют одинаковый размер, а разделы содержат одинаковое количество страниц

Если размер страницы и количество страниц в разделе выбрать равными степени двойки (2^k и 2^n соответственно), то принадлежность виртуального адреса к разделу и странице, а также смещение внутри страницы можно определить очень просто: **младшие k двоичных разрядов дают смещение, следующие n разрядов представляют собой номер виртуальной страницы, а оставшиеся старшие разряды (обозначим их количество через m) содержат номер раздела**

Двухуровневая страничная память

Для каждого раздела строится собственная таблица страниц

Количество дескрипторов в таблице и их размер подбираются такими, чтобы объем таблицы оказался равным объему страницы



Двухуровневая страничная память

Например, в процессоре Pentium при размере страницы 4 Кбайт длина дескриптора страницы составляет 4 байта, и количество записей в таблице страниц, помещающейся на странице, равняется соответственно 1024

Каждая таблица страниц описывается дескриптором, структура которого полностью совпадает со структурой дескриптора обычной страницы

Эти дескрипторы сведены в таблицу разделов, называемую также каталогом страниц. Физический адрес таблицы разделов активного процесса содержится в специальном регистре процессора и поэтому всегда известен операционной системе (CR3)

Страница, содержащая таблицу разделов, **никогда не выгружается из памяти**, в противном случае работа виртуальной памяти была бы невозможна

Двухуровневая страничная память

Выгрузка страниц с таблицами страниц позволяет сэкономить память, но при этом приводит к дополнительным временным затратам при получении физического адреса

Действительно, может случиться так, что та таблица страниц, которая содержит нужный дескриптор, в данный момент выгружена на диск, тогда процесс преобразования адреса приостанавливается до тех пор, пока требуемая страница не будет снова загружена в память

Для снижения вероятности отсутствия страницы в памяти используются различные приемы, основным из которых является кэширование

Двухуровневая страничная память

Проследим более подробно схему преобразования адресов для случая двухуровневой структуризации виртуального адресного пространства:

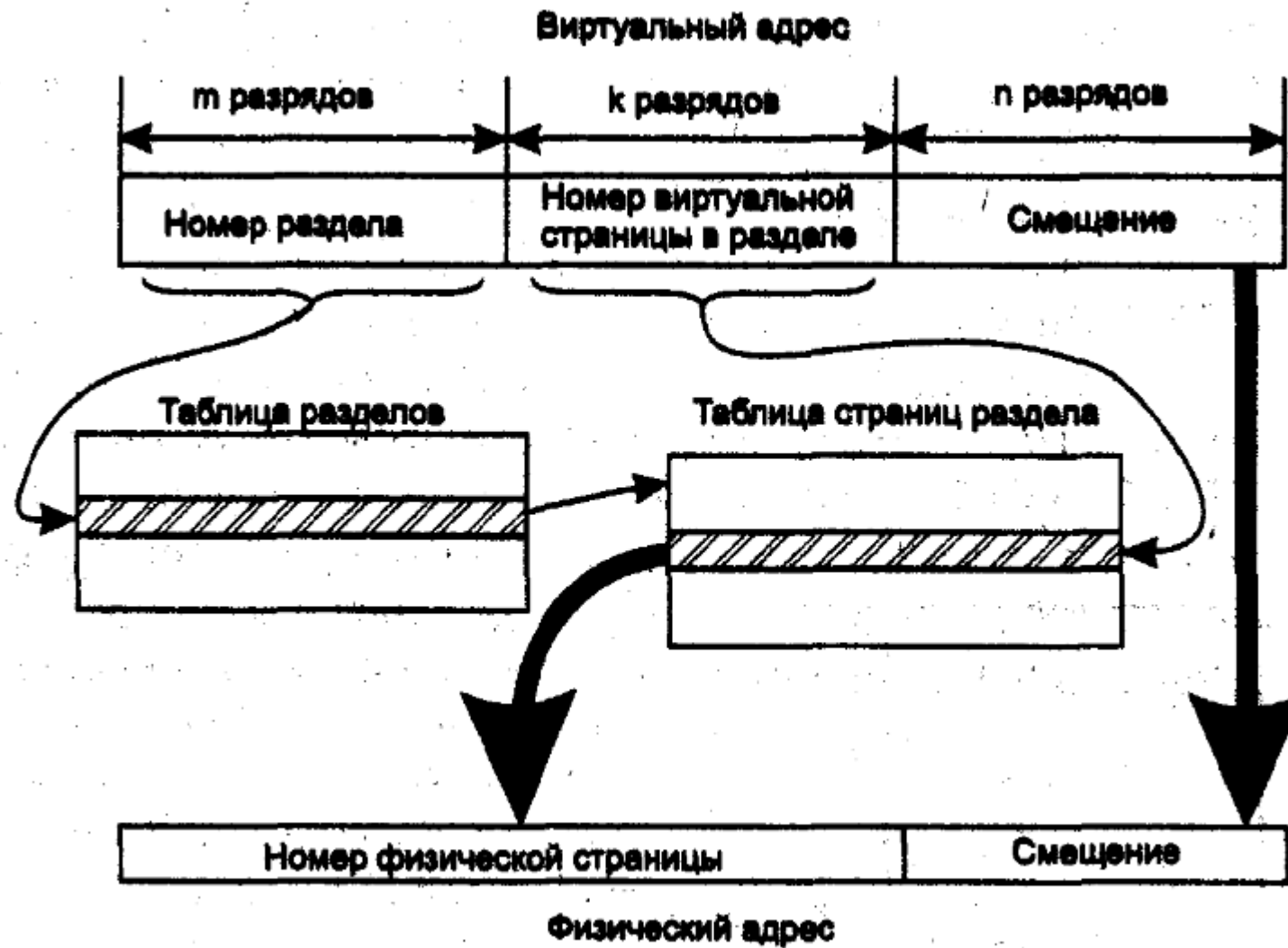
1. Путем отбрасывания $k + n$ младших разрядов в виртуальном адресе определяется номер раздела, которому принадлежит данный виртуальный адрес
2. По этому номеру из таблицы разделов извлекается дескриптор соответствующей таблицы страниц. Проверяется, находится ли данная таблица страниц в памяти. Если нет, происходит страничное прерывание, и система загружает нужную страницу с диска

Двухуровневая страничная память

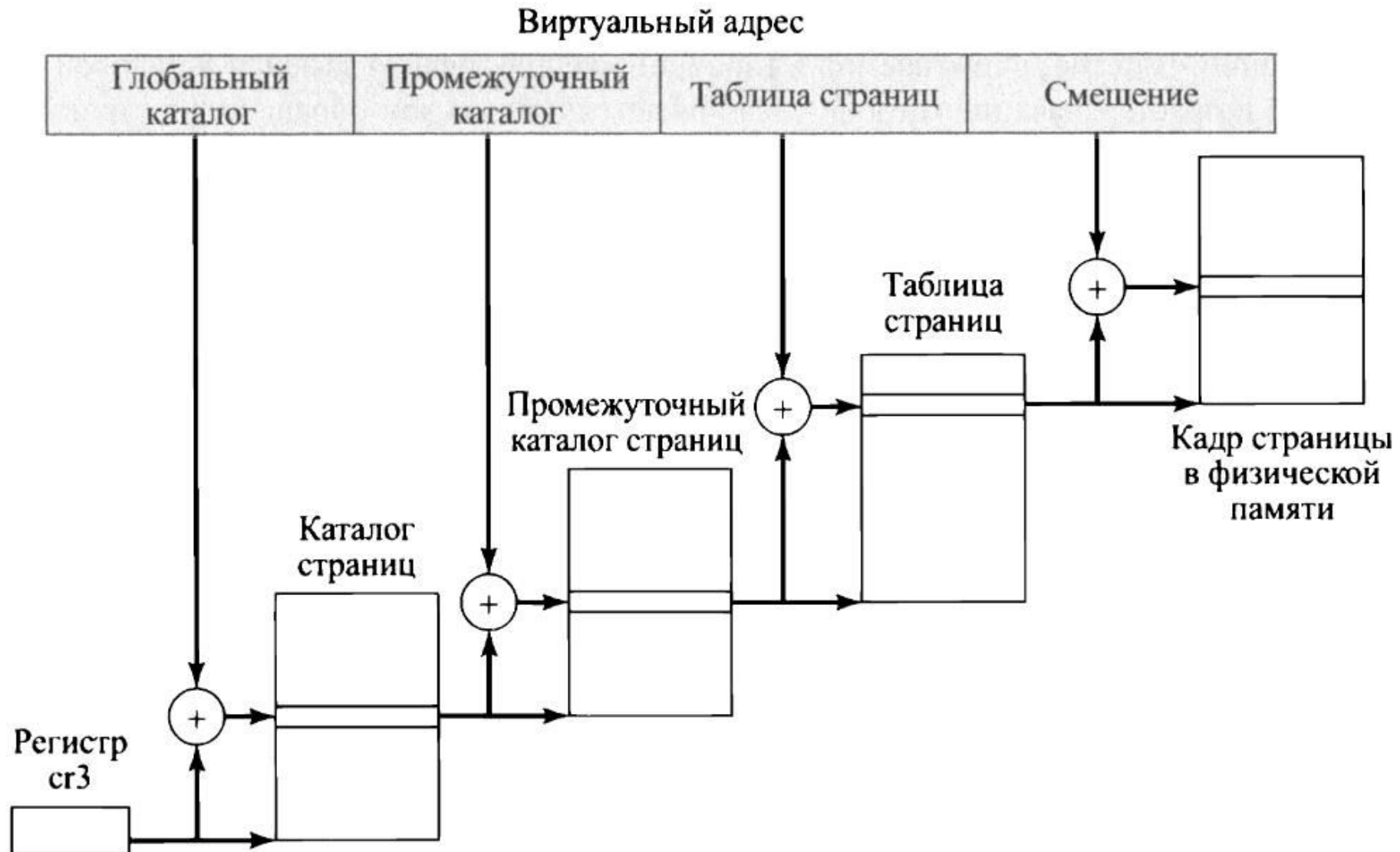
Далее из этой таблицы страниц извлекается дескриптор виртуальной страницы, номер которой содержится в средних n разрядах преобразуемого виртуального адреса. Снова выполняется проверка наличия данной страницы в памяти и при необходимости ее загрузка

4. Из дескриптора определяется номер (базовый адрес) физической страницы, в которую загружена данная виртуальная страница. К номеру физической страницы пристыковывается смещение, взятое из k младших разрядов виртуального адреса. В результате получается искомый физический адрес

Двухуровневая страничная память



Трехуровневая страничная память



Сегментное распределение

При страничной организации виртуальное адресное пространство процесса делится на равные части механически, без учета смыслового значения данных. В одной странице могут оказаться и коды команд, и инициализируемые переменные, и массив исходных данных программы

Такой подход не позволяет обеспечить дифференцированный доступ к разным частям программы, а это свойство могло бы быть очень полезным во многих случаях

Например, можно было бы запретить обращаться с операциями записи в сегмент программы, содержащий коды команд, разрешив эту операцию для сегментов данных

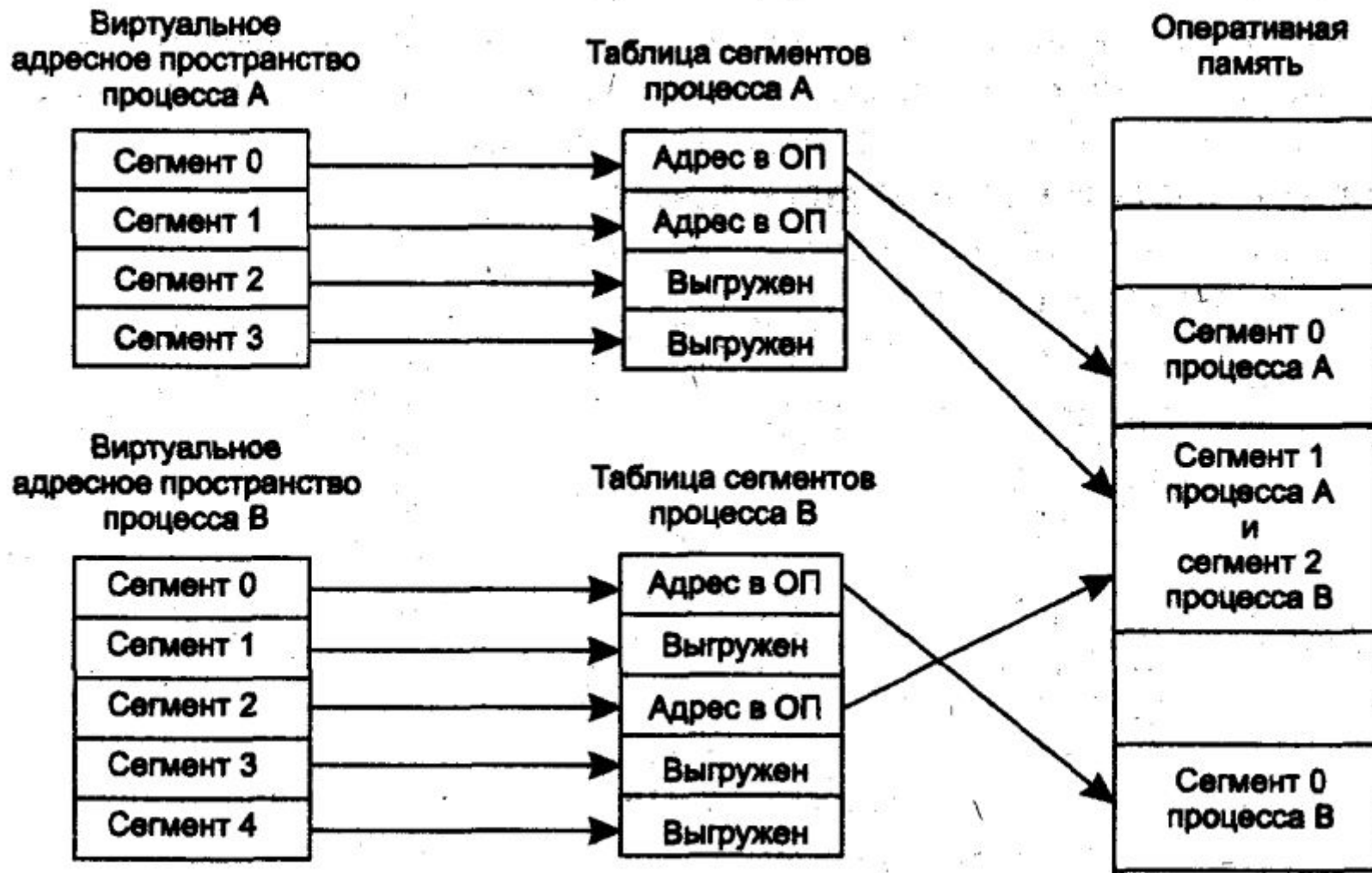
Сегментное распределение

Кроме того, разбиение виртуального адресного пространства на «осмысленные» части делает принципиально возможным совместное использование фрагментов программ разными процессами

Пусть, например, двум процессам требуется одна и та же подпрограмма, которая к тому же обладает свойством реентерабельности. Тогда коды этой подпрограммы могут быть оформлены в виде отдельного сегмента и включены в виртуальные адресные пространства обоих процессов. При отображении на физическую память сегменты, содержащие коды подпрограммы из обоих виртуальных пространств, проецируются на одну и ту же область физической памяти

Таким образом, оба процесса получают доступ к одной и той же копии подпрограммы

Сегментное распределение



Сегментное распределение

При **сегментной организации памяти** виртуальное адресное пространство процесса делится на части – **сегменты**, размер которых определяется с учетом смыслового значения содержащейся в них информации

Отдельный сегмент может представлять собой подпрограмму, массив данных и т. п. Деление виртуального адресного пространства на сегменты осуществляется компилятором на основе указаний программиста или по умолчанию, в соответствии с принятыми в системе соглашениями

Сегментное распределение

Максимальный размер сегмента определяется разрядностью виртуального адреса, например, при 32-разрядной организации процессора он равен 4 ГиБ

При этом максимально возможное виртуальное адресное пространство процесса представляет собой набор из N виртуальных сегментов, каждый размером по 4 ГиБ

В каждом сегменте виртуальные адреса находятся в диапазоне от 00000000_{16} до $FFFFFFFF_{16}$

Сегменты не упорядочиваются друг относительно друга, так что общего для сегментов линейного виртуального адреса не существует, виртуальный адрес задается парой чисел:

номером сегмента и линейным виртуальным адресом
внутри сегмента

Сегментное распределение

При загрузке процесса в оперативную память помещается только часть его сегментов, полная копия виртуального адресного пространства находится во внешней памяти

Для каждого загружаемого сегмента операционная система подыскивает непрерывный участок свободной памяти достаточного размера

Смежные в виртуальной памяти сегменты одного процесса могут занимать в оперативной памяти несмежные участки

Если во время выполнения процесса происходит обращение по виртуальному адресу, относящемуся к сегменту, который в данный момент отсутствует в памяти, то происходит прерывание

Сегментное распределение

ОС приостанавливает активный процесс, запускает на выполнение следующий процесс из очереди, а параллельно организует загрузку нужного сегмента с диска

При отсутствии в памяти места, необходимого для загрузки сегмента, операционная система выбирает сегмент на выгрузку, при этом она использует критерии, аналогичные рассмотренным ранее критериям выбора страниц при страничном способе управления памятью

На этапе создания процесса во время загрузки его образа в оперативную память система создает **таблицу сегментов** процесса (аналогичную таблице страниц)

Сегментное распределение

Каждая запись в таблице сегментов содержит следующие характеристики соответствующего сегмента:

- Базовый физический адрес сегмента в оперативной памяти
- Размер сегмента
- Правила доступа к сегменту
- Признаки модификации, присутствия и обращения к данному сегменту, а также некоторая другая информация

Если виртуальные адресные пространства нескольких процессов включают один и тот же сегмент, то в таблицах сегментов этих процессов делаются **ссылки на один и тот же участок** оперативной памяти, в который данный сегмент загружается в **единственном** экземпляре

Сегментное распределение

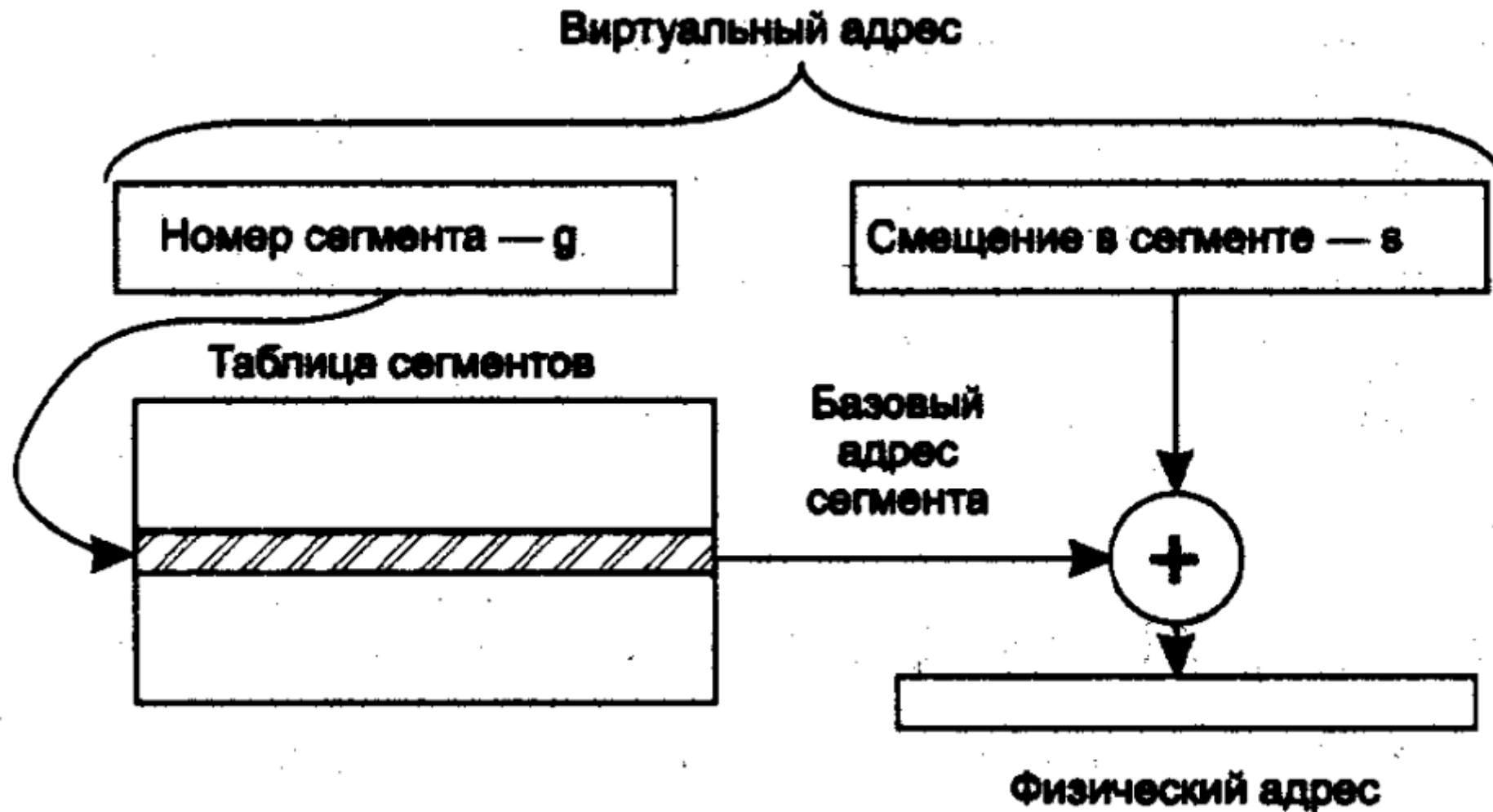
Как видно, сегментное распределение памяти имеет очень много общего со страничным распределением

Механизмы преобразования адресов этих двух способов управления памятью тоже весьма схожи, однако в них имеются и существенные отличия, которые являются следствием того, что сегменты, в отличие от страниц, имеют произвольный размер

Виртуальный адрес при сегментной организации памяти может быть представлен парой (g, s) , где g – номер сегмента, а s – смещение в сегменте

Физический адрес получается путем сложения базового адреса сегмента, который определяется по номеру сегмента g из таблицы сегментов и смещения s

Сегментное распределение



Сегментное распределение

В данном случае нельзя обойтись операцией конкатенации, как это делается при страничной организации памяти.

Действительно, поскольку размер страницы равен степени двойки, то в двоичном виде он выражается числом с несколькими нулями в младших разрядах

Страницы имеют одинаковый размер, а, значит, их начальные адреса кратны размеру страниц и выражаются также числами с нулями в младших разрядах.

Именно поэтому ОС заносит в таблицы страниц не полные адреса, а номера физических страниц, которые совпадают со старшими разрядами базовых адресов

Сегментное распределение

Сегмент же может в общем случае располагаться в физической памяти, начиная с любого адреса, следовательно, для определения местоположения в памяти необходимо задать его полный начальный физический адрес

Использование операции сложения вместо конкатенации **замедляет** процедуру преобразования виртуального адреса в физический по сравнению со страничной организацией

Другим недостатком сегментного распределения является **избыточность**, При сегментной организации единицей перемещения между памятью и диском является сегмент, имеющий в общем случае объем больший, чем страница

Сегментное распределение

Однако во многих случаях для работы программы вовсе не требуется загружать весь сегмент целиком, достаточно было бы одной или двух страниц. Аналогично, при отсутствии свободного места в памяти не стоит выгружать целый сегмент, когда можно обойтись выгрузкой нескольких страниц

Однако главный недостаток сегментного распределения – это **фрагментация**, которая возникает из-за непредсказуемости размеров сегментов

Сегментное распределение

Система с сегментной организацией функционирует аналогично системе со страничной организацией: при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический, время от времени происходят прерывания, связанные с отсутствием нужных сегментов в памяти, при необходимости освобождения памяти некоторые сегменты выгружаются

Одним из существенных отличий сегментной организации памяти от страничной является возможность задания

дифференцированных прав доступа процесса к его сегментам

Например, один сегмент данных, содержащий исходную информацию для приложения, может иметь права доступа «только чтение», а сегмент данных, представляющий результаты, – «чтение и запись»

Сегментно-страничное распределение

Сегментно-страничное распределение представляет собой комбинацию страничного и сегментного механизмов управления памятью и направлен на реализацию достоинств обоих подходов

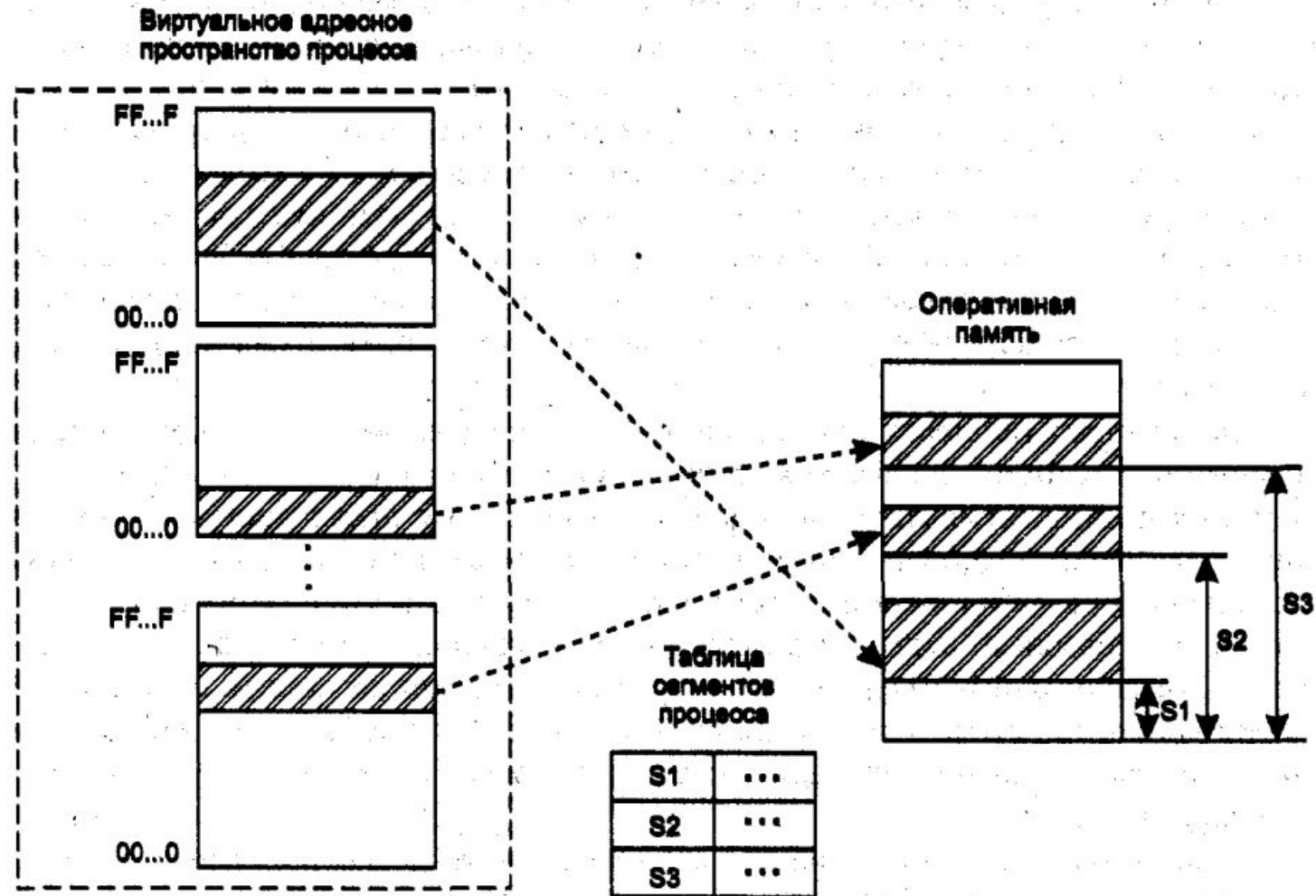
Так же как и при сегментной организации памяти, **виртуальное адресное пространство процесса делится на сегменты**. Это позволяет определять разные права доступа к разным частям кодов и данных программы

Перемещение данных между памятью и диском осуществляется **не сегментами, а страницами**. Для этого каждый виртуальный сегмент и физическая память делятся на страницы равного размера, что позволяет более эффективно использовать память, сократив до минимума фрагментацию

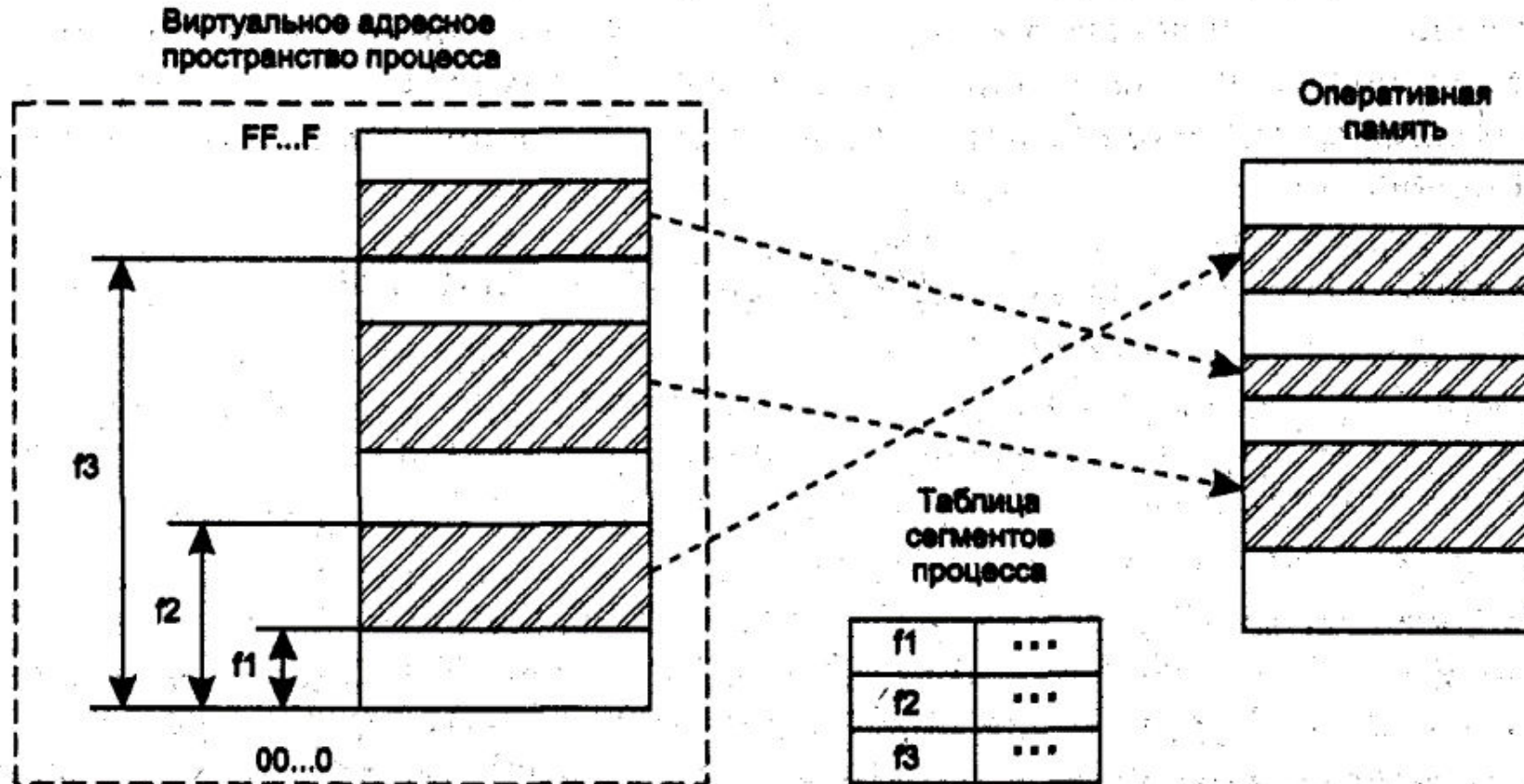
Сегментно-страничное распределение

В большинстве современных реализаций сегментно-страничной организации памяти, в отличие от набора виртуальных диапазонов адресов при сегментной организации памяти (а), все виртуальные сегменты образуют одно непрерывное линейное виртуальное адресное пространство (б)

Сегментно-страничное распределение



Сегментно-страничное распределение



Сегментно-страничное распределение

Координаты байта в виртуальном адресном пространстве при сегментно-страничной организации можно задать двумя способами

Во-первых, линейным виртуальным адресом, который равен сдвигу данного байта относительно границы общего линейного виртуального пространства

Во-вторых, – парой чисел, одно из которых является номером сегмента, а другое – смещением относительно начала сегмента

При этом в отличие от сегментной модели для однозначного задания виртуального адреса вторым способом необходимо каким-то образом указать также начальный виртуальный адрес сегмента с данным номером

Сегментно-страничное распределение

Системы виртуальной памяти ОС с сегментно-страничной организацией используют второй способ, так как он позволяет непосредственно определить принадлежность адреса некоторому сегменту и проверить права доступа процесса к нему

Для каждого процесса операционная система создает отдельную **таблицу сегментов**, в которой содержатся **описатели (дескрипторы) всех сегментов процесса**

Описание сегмента включает назначенные ему права доступа и другие характеристики, подобные тем, которые содержатся в дескрипторах сегментов при сегментной организации памяти

Сегментно-страничное распределение

Однако имеется и принципиальное отличие:

В поле базового адреса указывается не начальный физический адрес сегмента, отведенный ему в результате загрузки в оперативную память, а начальный линейный виртуальный адрес сегмента в пространстве виртуальных адресов (на двух прошлых схемах базовые физические адреса обозначены S_1 , S_2 , S_3 , а базовые виртуальные адреса – f_1 , f_2 , f_3)

Наличие базового виртуального адреса сегмента в дескрипторе позволяет однозначно преобразовать адрес, заданный в виде пары (номер сегмента, смещение в сегменте), в линейный виртуальный адрес байта, который затем преобразуется в физический адрес страничным механизмом

Сегментно-страничное распределение

Деление общего линейного виртуального адресного пространства процесса и физической памяти на страницы осуществляется так же, как это делается при страничной организации памяти

Размер страниц выбирается равным степени двойки, что упрощает механизм преобразования виртуальных адресов в физические

Виртуальные страницы нумеруются в пределах виртуального адресного пространства каждого процесса, а физические страницы – в пределах оперативной памяти

Сегментно-страничное распределение

Базовые адреса таблиц сегментов и страниц процесса являются частью его контекста. При активизации процесса эти адреса загружаются в специальные регистры процессора и используются механизмом преобразования адресов

Преобразование виртуального адреса в физический происходит в два этапа: **механизм сегментации** и **страничный механизм**

Сегментно-страничное распределение

На **первом этапе** работает **механизм сегментации**. Исходный виртуальный адрес, заданный в виде пары (номер сегмента, смещение), преобразуется в линейный виртуальный адрес

Для этого на основании базового адреса таблицы сегментов и номера сегмента вычисляется адрес дескриптора сегмента

Анализируются поля дескриптора и выполняется проверка возможности выполнения заданной операции. Если доступ к сегменту разрешен, то вычисляется линейный виртуальный адрес путем сложения базового адреса сегмента, извлеченного из дескриптора, и смещения, заданного в исходном виртуальном адресе

Сегментно-страничное распределение

На **втором этапе** работает **страничный механизм**.

Полученный линейный виртуальный адрес преобразуется в искомый физический адрес. В результате преобразования линейный виртуальный адрес представляется в том виде, в котором он используется при страничной организации памяти, а именно в виде пары (номер страницы, смещение в странице)

Благодаря тому, что размер страницы выбран равным степени двойки, эта задача решается простым отделением некоторого количества младших двоичных разрядов

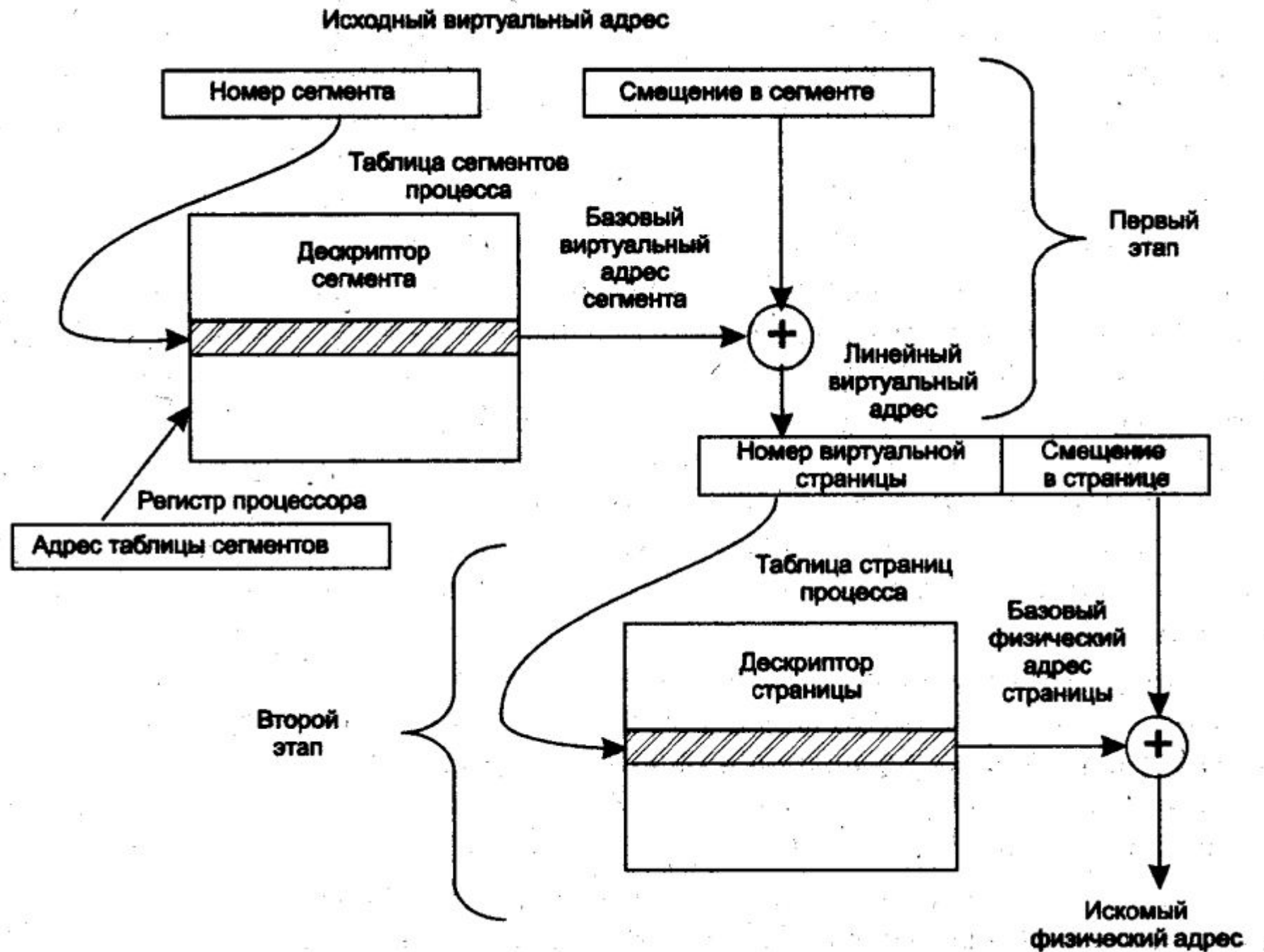
В результате в старших разрядах содержится номер виртуальной страницы, а в младших – смещение искомого элемента относительно начала страницы

Сегментно-страничное распределение

Так, если размер страницы равен 2^k , то смещением является содержимое младших k разрядов, а остальные, старшие, разряды содержат номер виртуальной страницы, которой принадлежит искомый адрес

Далее преобразование адреса происходит так же, как при страничной организации: старшие разряды линейного виртуального адреса, содержащие номер виртуальной страницы, заменяются номером физической страницы, взятым из таблицы страниц, а младшие разряды виртуального адреса, содержащие смещение, остаются без изменений

Сегментно-страничное распределение



Сегментно-страничное распределение

Как видно, механизм сегментации и страничный механизм действуют достаточно независимо друг от друга. Поэтому нетрудно представить себе реализацию сегментно-страничного управления памятью, в которой механизм сегментации работает по вышеописанной схеме, а страничный механизм изменен в соответствии с двухуровневой схемой

В таком случае преобразование виртуального адреса в физический происходит в несколько этапов:

- Сначала механизм сегментации обычным образом, используя таблицу сегментов, вычисляет линейный виртуальный адрес
- Затем из данного виртуального адреса вычленяются номера раздела и страницы, а также смещение
- Далее по номеру раздела из таблицы разделов определяется адрес таблицы страниц
- Затем по номеру виртуальной страницы из таблицы страниц определяется номер физической страницы, к которому пристыковывается смещение

Сегментно-страничное распределение

Рассмотрим еще одну возможную схему управления памятью, основанную на комбинировании сегментного и страничного механизмов. Так же как и в предыдущих случаях, виртуальное пространство процесса делится на сегменты, а каждый сегмент, в свою очередь, делится на виртуальные страницы.

Первое отличие состоит в том, что виртуальные страницы нумеруются не в пределах всего адресного пространства процесса, а в пределах сегмента

Виртуальный адрес в этом случае выражается тройкой (номер сегмента, номер страницы, смещение в странице)

Сегментно-страничное распределение

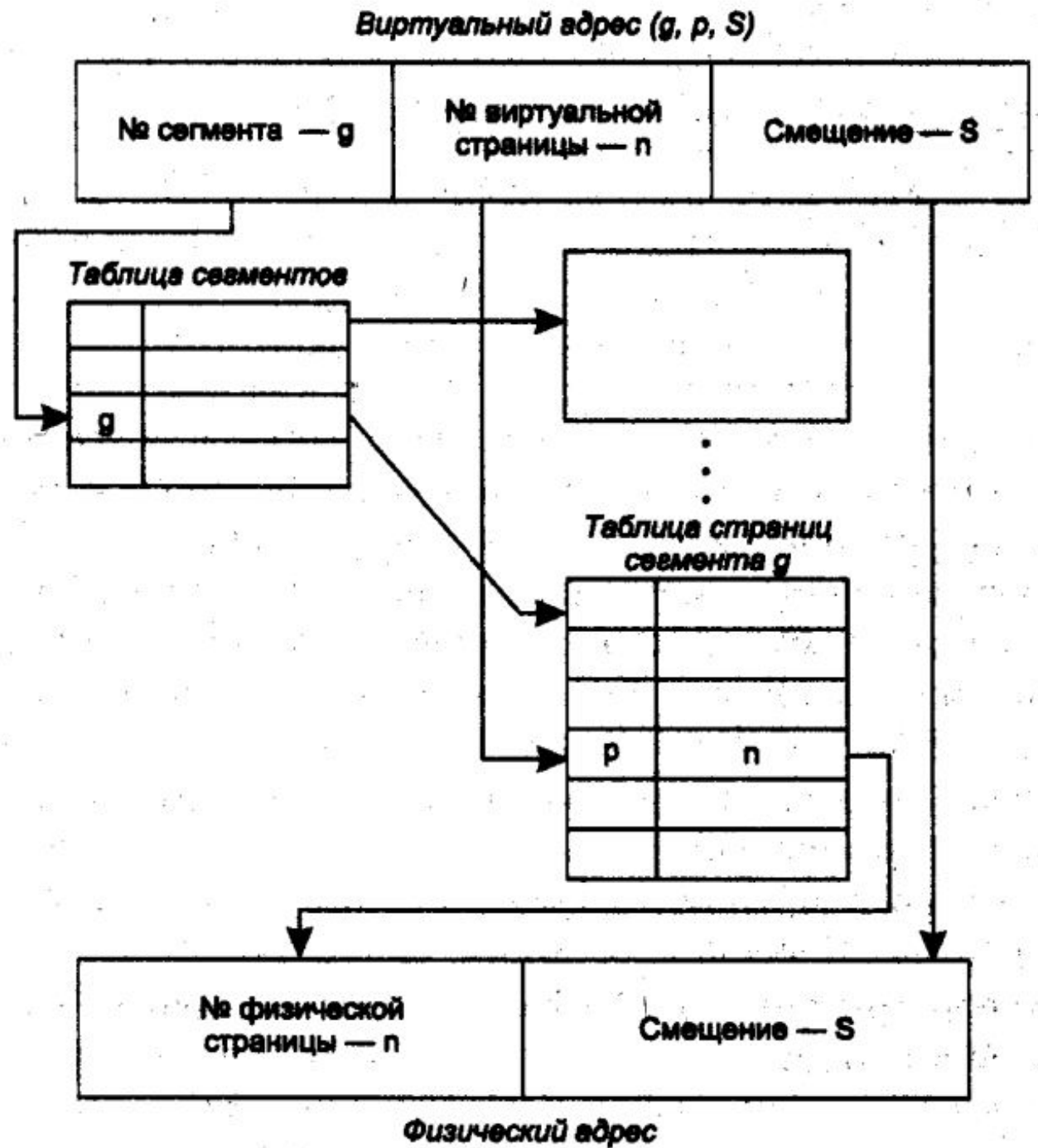
Загрузка процесса выполняется операционной системой постранично, при этом часть страниц размещается в оперативной памяти, а часть – на диске

Для каждого процесса создается собственная таблица сегментов, а для каждого сегмента – своя таблица страниц.

Адрес таблицы сегментов загружается в специальный регистр процессора, когда активизируется соответствующий процесс

Таблица страниц содержит дескрипторы страниц, содержимое которых полностью аналогично содержимому описанных ранее дескрипторов страниц

Сегментно-страничное распределение



Сегментно-страничное распределение

1. По номеру сегмента, заданному в виртуальном адресе, из таблицы сегментов извлекается физический адрес соответствующей таблицы страниц
2. По номеру виртуальной страницы, заданному в виртуальном адресе, из таблицы страниц извлекается дескриптор, в котором указан номер физической страницы
3. К номеру физической страницы пристыковывается младшая часть виртуального адреса – смещение

Разделяемые сегменты памяти

Хотя основной задачей операционной системы при управлении памятью является защита областей оперативной памяти, принадлежащей одному из процессов, от доступа к ней остальных процессов, в некоторых случаях оказывается полезным организовать **контролируемый совместный доступ нескольких процессов к определенной области памяти**

Например, в том случае, когда несколько пользователей одновременно работают с некоторым текстовым редактором, нецелесообразно многократно загружать его код в оперативную память

Разделяемые сегменты памяти

Гораздо экономичней загрузить всего одну копию кода, которая обслуживала бы всех пользователей, работающих в данное время с этим редактором (для этого код редактора должен быть реентерабельным)

Очевидно, что сегмент данных редактора не может присутствовать в памяти в единственном разделяемом экземпляре – для каждого пользователя должна быть создана своя копия этого сегмента, в которой помещаются редактируемый текст и значения других переменных редактора, например его конфигурация, индивидуальная для каждого пользователя, и т. п.

Разделяемые сегменты памяти

Другим примером применения разделяемой области памяти может быть использование ее в качестве буфера при межпроцессном обмене данными

В этом случае один процесс пишет в разделяемую область, а другой – читает

Подсистема виртуальной памяти представляет собой удобный механизм для решения задачи совместного доступа нескольких процессов к одному и тому же сегменту памяти, который в этом случае становится разделяемым, а сама память называется **разделяемой (shared memory)**

Разделяемые сегменты памяти

Для организации разделяемого сегмента при наличии подсистемы виртуальной памяти достаточно поместить его в виртуальное адресное пространство каждого процесса, которому нужен доступ к данному сегменту, а затем настроить параметры отображения этих виртуальных сегментов так, чтобы они соответствовали одной и той же области оперативной памяти

Детали такой настройки зависят от типа используемой в ОС модели виртуальной памяти: сегментной или сегментно-страничной (чисто страничная организация не поддерживает понятие «сегмент», что делает невозможным решение рассматриваемой задачи)

Разделяемые сегменты памяти

Например, при сегментной организации необходимо в дескрипторах виртуального сегмента каждого процесса указать один и тот же базовый физический адрес

При сегментно-страничной организации отображение на одну и ту же область памяти достигается за счет соответствующей настройки таблицы страниц каждого процесса

В приведенном описании подразумевается, что разделяемый сегмент помещается в индивидуальную часть виртуального адресного пространства каждого процесса (а) и описывается в каждом процессе индивидуальным дескриптором сегмента (и индивидуальными дескрипторами страниц, если используется сегментно-страничный механизм)

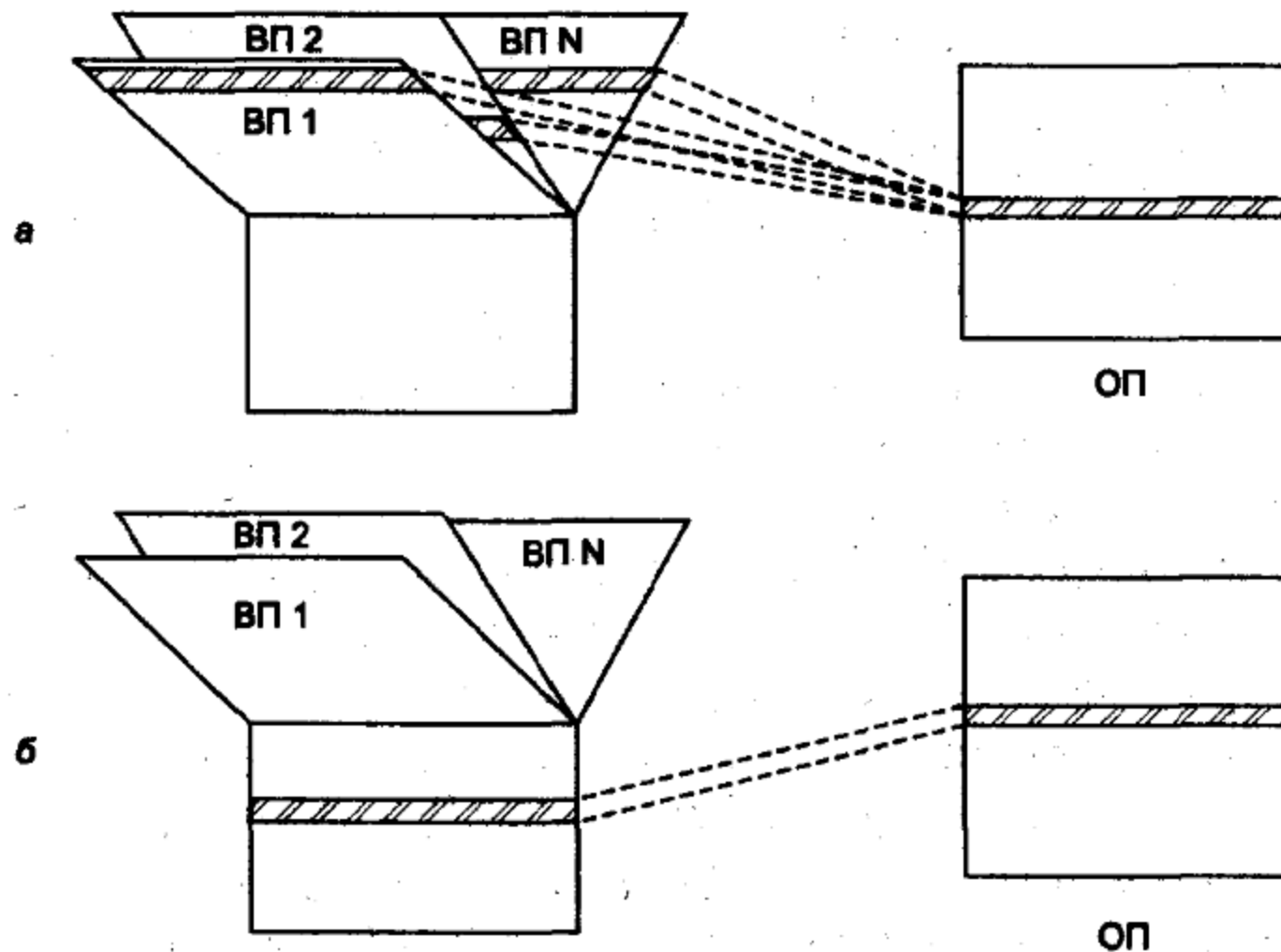
Разделяемые сегменты памяти

«Попадание» же этих виртуальных сегментов в общую часть оперативной памяти достигается за счет согласованной настройки операционной системой многочисленных дескрипторов для множества процессов

Есть и более экономичное для ОС решение этой задачи – помещение единственного разделяемого виртуального сегмента в общую часть виртуального адресного пространства процессов, то есть в ту часть, которая обычно используется для модулей ОС (б)

В этом случае настройка дескриптора сегмента (и дескрипторов страниц) выполняется только один раз, а все процессы на основе такой настройки совместно используют часть оперативной памяти

Разделяемые сегменты памяти



Разделяемые сегменты памяти

При работе с разделяемыми сегментами памяти ОС должна выполнять некоторые функции, общие для любых разделяемых между процессами ресурсов – файлов, семафоров и т. п.

Эти функции состоят в поддержке схемы именования ресурсов, проверке прав доступа определенного процесса к ресурсу, а также в отслеживании количества процессов, пользующихся данным ресурсом (чтобы удалить его в случае ненужности)

Для того чтобы отличать разделяемые сегменты памяти от индивидуальных, дескриптор сегмента должен содержать поле, имеющее два значения: **разделяемый (shared)** и **индивидуальный (private)**

Разделяемые сегменты памяти

Операционная система может создавать разделяемые сегменты как по явному запросу, так и по умолчанию

В первом случае прикладной процесс должен выполнить соответствующий системный вызов, по которому операционная система создает новый сегмент в соответствии с указанными в вызове параметрами: размером сегмента, разрешенными над ним операциями (чтение/запись) и идентификатором

Все процессы, выполнившие подобные вызовы с одними тем же идентификатором, получают доступ к этому сегменту и используют его по своему усмотрению, например, в качестве буфера для обмена данными

Разделяемые сегменты памяти

Во втором случае операционная система сама в определенных ситуациях принимает решение о том, что нужно создать разделяемый сегмент

Наиболее типичным примером такого рода является поступление нескольких запросов на выполнение одного и того же приложения. Если кодовый сегмент приложения помечен в исполняемом файле как реентерабельный и разделяемый, то ОС не создает при поступлении нового запроса новую индивидуальную для процесса копию кодового сегмента этого приложения, а отображает уже существующий разделяемый сегмент на виртуальное адресное пространство процесса

Разделяемые сегменты памяти

При закрытии приложения каким-либо процессом ОС проверяет, существуют ли другие процессы, пользующиеся данным приложением, и если их нет, то удаляет данный разделяемый сегмент

Разделяемые сегменты выгружаются на диск системой виртуальной памяти по тем же алгоритмам и с помощью тех же механизмов, что и индивидуальные

Операционные системы

Лекция 13

Свопинг и виртуальная память